Лабораторная работа 2:
Продвинутые методы безусловной оптимизации.

1 Алгоритмы

В этом задании все рассматриваемые алгоритмы будут решать общую задачу нелинейной безусловной оптимизации:

$$\min_{x \in \mathbb{R}^n} f(x)$$

где функция $f : \mathbb{R}^n \to \mathbb{R}$ является достаточно гладкой.

1.1 Метод сопряженных градиентов

Стандартный метод сопряженных градиентов является итерационным методом (неточного) решения системы линейных уравнений

$$Ax = b$$

где $A \in \mathbb{S}^n_{++}$ и $b, x \in \mathbb{R}^n$.

Обозначим невязку на k-м шаге через $g_k := Ax_k - b$.

Итерация метода:

$$x_{k+1} = x_k + \frac{\langle g_k, g_k \rangle}{\langle Ad_k, d_k \rangle} d_k.$$

Направления пересчитываются по правилу:

$$d_{k+1} = -g_{k+1} + \frac{\langle g_{k+1}, g_{k+1} \rangle}{\langle g_k, g_k \rangle} d_k,$$

где $d_0 := -g_0$.

Заметим, что методу сама матрица A не нужна, а нужна процедура умножения этой матрицы на произвольный вектор. Это является одним из основных преимуществ метода над точными процедурами типа метода Гаусса или разложения Холецкого.

Важно: В эффективной реализации метода должно быть лишь одно матрично-векторное произведение за итерацию.

В качестве критерия остановки обычно используют тест

$$\|Ax_k - b\| \leq \epsilon \|b\|.$$

Для произвольной неквадратичной функции $f(x)$ применяется нелинейный вариант метода. Направление пересчитывается по правилу $d_{k+1} = -g_{k+1} + \beta_k d_k$. Коэффициент β_k вычисляется по одной из классических формул, например Флетчера-Ривса (FR) или Полака-Рибьера (PR):

$$\beta_k^{FR} = \frac{||g_{k+1}||^2}{||g_k||^2}, \quad \beta_k^{PR} = \frac{\langle g_{k+1}, g_{k+1} - g_k \rangle}{||g_k||^2}$$

Для нелинейного метода критически важно использовать линейный поиск, удовлетворяющий строгим условиям Вульфа. Также необходимо применять эвристику рестарта: если $\beta_k < 0$ или метод перестал давать направление убывания, следует сделать сброс $\beta_k = 0$ (шаг обычного градиентного спуска).

1.2 Усеченный метод Ньютона

Напомним, что обычный (демпфированный) метод Ньютона итеративно строит последовательность точек $(x_k)_{k=0}^{\infty}$ по следующему правилу:

$$x_{k+1} = x_k + \alpha_k d_k$$

где направление поиска d_k находится из следующей системы линейных уравнений:

$$\nabla^2 f(x_k) d_k = -\nabla f(x_k). \tag{1}$$

Здесь $\nabla^2 f(x_k) \in \mathbb{S}^n_{++}$– гессиан[1] функции f в точке x_k. Длина шага α_k выбирается с помощью линейного поиска.

В усеченном методе Ньютона[2] система (1) решается с помощью метода сопряженных градиентов. В этом случае сама матрица $\nabla^2 f(x_k)$ методу не нужна, а нужна только процедура умножения матрицы $\nabla^2 f(x_k)$ на произвольный вектор $v \in \mathbb{R}^n$. Кроме того, не имеет большого смысла решать систему (1) очень точно, если текущая точка x_k находится далеко от оптимума. Поэтому обычно метод сопряженных градиентов останавливают, как только невязка $r_k(d) := \nabla^2 f(x_k) d + \nabla f(x_k)$ удовлетворяет условию

$$\|r_k(d)\| \leq \eta_k \|\nabla f(x_k)\|. \tag{2}$$

Последовательность $\eta_k \in (0, 1)$, называется форсирующей последовательностью и обычно выбирается следующим образом:

$$\eta_k := \min\left\{0.5, \sqrt{\|\nabla f(x_k)\|}\right\}$$

Такой выбор гарантирует локальную сверхлинейную скорость сходимости метода в невырожденном случае.

Следует отметить, что ранний выход из метода сопряженных градиентов по условию (2), вообще говоря, не гарантирует того, что найденное направление d_k будет направлением спуска функции f в точке x_k (в отличие от

[1] Здесь предполагается, что гессиан $\nabla^2 f(x_k)$ является положительно определенным. Одно из достаточных условий для этого - строгая выпуклость функции f. Если гессиан $\nabla^2 f(x_k)$ оказался не положительно определенным, то обычно выполняют его модификацию. В этом задании мы не будем рассматривать модификации гессиана.

[2] В литературе этот метод также известен как неточный метод Ньютона или безгессианный метод Ньютона.

точного решения $-\left[\nabla^2 f\left(x_k\right)\right]^{-1} \nabla f\left(x_k\right)$ системы (1)). Поэтому после выхода из метода сопряженных градиентов нужно дополнительно проверять, удовлетворяет ли d_k условию $\left\langle \nabla f\left(x_k\right), d_k\right\rangle < 0$, и если нет, то снова запустить метод сопряженных градиентов, но теперь уже из начальной точки d_k и с меньшим значением η_k (например, уменьшить η_k в 10 раз). Такую процедуру нужно повторять до тех пор, пока d_k не станет направлением спуска (что гарантированно рано или поздно произойдет).

Внимание: для консистентности реализаций в качестве начального приближения в методе сопряжённых градиентов (аргумент $_0$) рекомендуется использовать точку $-\nabla f\left(x_k\right)$

Заметим, что в усеченном методе Ньютона, так же, как и в обычном методе Ньютона, важно начинать линейный поиск с $\alpha_k^{(0)} = 1$. В противном случае никакой локальной сверхлинейной сходимости может и не быть.

1.3 Метод L-BFGS

Метод BFGS принадлежит классу квазинъютоновских методов, которые на каждом шаге аппроксимируют настоящий гессиан $\nabla^2 f\left(x_k\right)$ с помощью некоторой матрицы B_k и выбирают направление спуска d_k как решение следующей системы (аналогичной ньютоновской):

$$B_k d_k = -\nabla f\left(x_k\right) \quad \Leftrightarrow \quad d_k = -H_k \nabla f\left(x_k\right), \qquad \text{где } H_k := B_k^{-1}.$$

Дальше, из текущей точки x_k, как обычно, выполняется шаг в этом направлении:

$$x_{k+1} = x_k + \alpha_k d_k$$

где $\alpha_k > 0$ - длина шага, настраиваемая с помощью линейного поиска.

Основная работа на каждой итерации квазиньютоновского метода затрачивается на построение аппроксимации гессиана и вычисление направления поиска.

Начиная с $B_0 = I$, алгоритм пересчитывает аппроксимацию гессиана по правилу $B_{k+1} = B_k + U_k$, где $U_k -$ некоторое низкоранговое обновление. Маленький ранг U_k необходим для построения эффективной процедуры вычисления обратной матрицы $H_{k+1} = B_{k+1}^{-1} = \left(B_k + U_k\right)^{-1}$. Конкретный вид обновления U_k следует из выполнения нескольких требований. Основное из них - уравнение секущей: $B_{k+1}\left(x_{k+1} - x_k\right) = \nabla f\left(x_{k+1}\right) - \nabla f\left(x_k\right)$, справедливое для всех квазиньютоновских методов. Одного этого уравнения недостаточно, чтобы однозначно определить B_{k+1}. Конкретную квазиньютоновскую схему получают с помощью наложения на аппроксимацию гессиана дополнительных требований.

Наиболее популярным и устойчивым на практике является правило BFGS (по фамилиям авторов: Бройден-Флетчер-Гольдфарб-Шанно). Обозначим:

$$\begin{aligned} s_k &:= x_{k+1} - x_k, \\ y_k &:= \nabla f\left(x_{k+1}\right) - \nabla f\left(x_k\right). \end{aligned}$$

Тогда, схема пересчета BFGS имеет следующий вид:

$$
\begin{aligned}
B_{k+1} &= B_k - \frac{B_k s_k s_k^T B_k}{\langle B_k s_k, s_k \rangle} + \frac{y_k y_k^T}{\langle y_k, s_k \rangle} \\
H_{k+1} &= \left(I_n - \frac{s_k y_k^T}{\langle y_k, s_k \rangle} \right) H_k \left(I_n - \frac{y_k s_k^T}{\langle y_k, s_k \rangle} \right) + \frac{s_k s_k^T}{\langle y_k, s_k \rangle}
\end{aligned} \tag{3}
$$

Метод L-BFGS является модификацией метода BFGS для случаев, когда не удаётся поместить матрицу H_k в память. Для этого на каждой итерации метода поддерживается история $\mathcal{H}_k := ((s_{k-i}, y_{k-i}))_{i=1}^{l}$ из последних l векторов, где l - некоторый параметр (типичное значение $l = 10$; при $k < l$ история $\mathcal{H}_k$ состоит только из k пар.) Далее в качестве матрицы H_k выбирается матрица, полученная с помощью l-кратного рекуррентного применения формулы обновления (3), где в качестве начальной матрицы выбирается

$$
H_{k-l} := \gamma_0^{(k)} I_n, \qquad \text{где } \gamma_0^{(k)} := \frac{\langle y_{k-1}, s_{k-1} \rangle}{\langle y_{k-1}, y_{k-1} \rangle} \tag{4}
$$

Таким образом, метод L-BFGS является усеченной версией BFGS со специальном выбором начальной матрицы по формуле (4) (так называемое правило Барзилая-Борвейна).

Преимуществом такой схемы является то, что для ее реализации никаких матриц хранить в памяти не требуется. Действительно, напомним, что методу сама матрица H_k не нужна, а нужен вектор $d_k = -H_k \nabla f(x_k)$. Таким образом, нужна процедура, вычисляющая результат умножения L-BFGS матрицы H_k на произвольный вектор $v \in \mathbb{R}^n$. Умножая обе части (3) на v, видно, что такую процедуру можно организовать рекурсивно без явного формирования каких-либо матриц в памяти:

Алгоритм 1 Рекурсивное умножение L-BFGS матрицы на вектор

```
function BFGS_Multiply(v, H, γ0)
    if H = ∅ then
        return γ0 v
    end if
    (s, y) ← последняя пара из H.
    H' ← H без последней пары.
    v' ← v − ⟨s,v⟩/⟨y,s⟩ y
    z ← BFGS_Multiply(v', H', γ0)
    return z + (⟨s,v⟩ − ⟨y,z⟩)/⟨y,s⟩ s.
end function
```

Имея в распоряжении указанную процедуру, направление d_k вычислить легко:

Алгоритм 2 Вычисление направления поиска d_k в методе L-BFGS

```
function LBFGS_direction
    (s, y) ← последняя пара из H_k
    γ0 ← ⟨y,s⟩/⟨y,y⟩
    return BFGS_Multiply(−∇f(x_k), H_k, γ0)
end function
```

1.4 Разностная проверка произведения гессиана на вектор

Проверить правильность подсчета произведения гессиана $\nabla^2 f(x) \in \mathbb{S}^n$ на заданный вектор $v \in \mathbb{R}^n$ можно с помощью конечных разностей:

$$\left[\nabla^2 f(x) v\right]_i \approx \frac{f\left(x + \epsilon_2 v + \epsilon_2 e_i\right) - f\left(x + \epsilon_2 v\right) - f\left(x + \epsilon_2 e_i\right) + f(x)}{\epsilon_2^2},$$

где $e_i := (0, \ldots, 0, 1, 0, \ldots, 0)$ – i-й базисный орт, а $\epsilon_2 \sim \sqrt[3]{\epsilon_{\text{mach}}}$, где ϵ_{mach}– машинная точность $\left(\approx 10^{-16}\right.$ для типа double).

2 Формулировка задания

Как и в первой лабораторной работе, задание выполняется в командах по 3 человека. Каждая команда проводит базовые эксперименты на ML-оракулах из своего варианта (закрепленного с Лабораторной №1) и выполняет индивидуальный исследовательский трек по вариантам из Приложения A.

В отчете необходимо явно указать, как были распределены задачи между участниками команды.

2.1 План выполнения

1. Даны прототипы функций, которые Вам нужно будет реализовать. Некоторые процедуры уже частично или полностью реализованы. Обратите внимание, что реализацию классов линейного поиска и оракулов Вы можете взять из Вашей реализации первого практического задания.

2. Реализуйте метод сопряженных градиентов для решения системы линейных уравнений (функция Linear_conjugate_gradients в модуле optimization). Этот метод будет использоваться как внутренний решатель HFN.

3. Реализуйте нелинейный метод сопряженных градиентов (функция nonlinear_conjugate_gradients) для минимизации произвольных функций. В качестве базовой формулы использовать Полака-Рибьера (PR). Для этого метода реализовать линейный поиск по строгим условиям Вульфа и эвристику рестарта.

4. Для ML-оракулов (в модуле oracles) реализуйте метод hess_vec, выполняющий умножение гессиана на заданный вектор.

Замечание: Ваш код должен поддерживать как плотные матрицы A типа np.array, так и разреженные типа scipy.sparse.csr_matrix.

5. Реализуйте подсчет разностной аппроксимации произведения гессиана на заданный вектор (функция hess_vec_finite_diff в модуле oracles). С помощью реализованной процедуры проверьте правильность реализации метода hess_vec ML-оракулов. Для этого сгенерируйте небольшую модельную выборку и сравните значения, выдаваемые методом hess_vec, с соответствующими разностными аппроксимациями в нескольких пробных точках.

6. Реализуйте усеченный метод Ньютона (функция hessian_free_newton в модуле optimization). При реализации этого метода используйте свою реализацию метода сопряженных градиентов (функция linear_conjugate_gradients).

7. Реализуйте метод L-BFGS (функция lbfgs в модуле optimization).

8. Проведите эксперименты, описанные ниже. Напишите отчет.

2.2 Эксперимент: Зависимость числа итераций метода сопряженных градиентов от числа обусловленности и размерности пространства

Проведите эксперимент, аналогичный соответствующему эксперименту с градиентным спуском из первого практического задания. Сравните результат метода сопряженных градиентов с результатом градиентного спуска. Какие выводы можно сделать?

2.3 Эксперимент: Выбор размера истории в методе L-BFGS

2.3.1 Используемые функции:

Ваши индивидуальные ML-оракулы (задача классификации и задача регрессии) на соответствующих датасетах из LIBSVM. Коэффициент регуляризации выберите стандартным образом: $\lambda = 1/m$.

2.3.2 Порядок проведения:

Исследуйте, как влияет размер истории в методе L-BFGS на поведение метода.

Прежде всего, оцените размер требуемой памяти и сложность итерации метода L-BFGS в зависимости от размера истории l и размерности пространства n. (Здесь не нужно учитывать сложность оракула.)

Рассмотрите несколько вариантов вариантов выбора размера истории (например, $l = 0, l = 1, l = 5,\ l = 10, l = 50, l = 100$). Примечание: случай $l = 0$ эквивалентен градиентному спуску с линейным поиском Вульфа. Постройте следующие графики

(a) Зависимость относительного квадрата нормы градиента $\|\nabla f(x_k)\|^2 / \|\nabla f(x_0)\|^2$ (в логарифмической шкале) против номера итерации.

(b) Зависимость относительного квадрата нормы градиента $\|\nabla f(x_k)\|^2 / \|\nabla f(x_0)\|^2$ (в логарифмической шкале) против реального времени работы.

При этом разные варианты выбора размера истории нужно рисовать на одном и том же графике.

Коэффициент регуляризации и начальную точку выберите стандартным образом: $\lambda = 1/m, x_0 = 0$.

2.3.3 Вопросы для анализа в отчете:

1. Наблюдаете ли вы «плато», после которого увеличение L почти не сокращает количество итераций? Как это значение L соотносится с размерностью задачи n?

2. Постройте график: по оси X – размер истории L, по оси Y – итоговое время работы. Найдите «золотую середину» (оптимальное L) для вашего датасета. Почему слишком большое L может замедлять метод, даже если итераций становится меньше?

3. Как вы думаете, если бы признаки в датасете были сильно скоррелированы (плохая обусловленность), потребовалось бы нам большее или

меньшее значение L для эффективной работы. Проиллюстрируйте это на плохо обусловленной квадратичной функции из Лабораторной работы 1.

4. Насколько сильно учет даже минимальной истории ($l = 1$ или $l = 5$ ускоряет сходимость по сравнению с обычным градиентным спуском $l = 0$).

2.4 Эксперимент: Сравнение эффективности методов на реальных задачах машинного обучения

2.4.1 Используемые функции:

Ваши индивидуальные ML-оракулы (задача классификации и задача регрессии) на соответствующих датасетах из LIBSVM. Коэффициент регуляризации выберите стандартным образом: $\lambda = 1/m$.

2.4.2 Порядок проведения:

Запустите нелинейный метод CG, усеченный метод Ньютона, метод L-BFGS из начальной точки $x_0 = 0$. Для L-BFGS используйте размер истории $L = 10$. Для усеченного Ньютона используйте адаптивный критерий остановки внутреннего цикла (форсирующую последовательность $\eta_k = \min(0.5, \sqrt{\nabla f(x_k)})$. Для всех методов используйте относительный критерий остановки по норме градиента.

Постройте три семейства графиков:

(a) Зависимость значения функции против номера итерации метода.

(b) Зависимость значения функции против реального времени работы.

(c) Зависимость относительного квадрата нормы градиента $\|\nabla f(x_k)\|^2 / \|\nabla f(x_0)\|^2$ (в логарифмической шкале) против реального времени работы.

При этом все методы нужно рисовать на одном и том же графике.

2.4.3 Вопросы для анализа в отчете:

1. Какой метод делает меньше всего внешних итераций? Совпадает ли лидер по числу итераций с лидером по реальному времени работы? Почему усеченный Ньютон может проигрывать L-BFGS в секундах, даже если ему требуется в 10 раз меньше итераций?

2. Как ведут себя методы на «широких» данных ($n \gg m$) по сравнению с «высокими» ($m \gg n$)? Какой метод оказался наиболее устойчивым к размерности пространства?

3. Добавьте на графики кривые обычного Градиентного спуска и полного метода Ньютона из Лабораторной работы 1. Насколько велик разрыв в производительности между методами первого порядка и продвинутыми алгоритмами этой лабораторной работы?

2.5 Эксперимент: Микропрофилирование вычислительных затрат

2.5.1 Используемые функции:

Ваши индивидуальные ML-оракулы (задача классификации и задача регрессии) на соответствующих датасетах из LIBSVM. Коэффициент регуляризации выберите стандартным образом: $\lambda = 1/m$.

2.5.2 Порядок проведения:

Для каждого из трех продвинутых методов замерьте, куда именно уходит время внутри одной итерации оптимизации. Постройте столбчатые диаграммы, разбивающие время 1 итерации на 3 компоненты:

- Оракул: вычисление ∇f (для всех) и $\nabla^2 f \cdot v$ (для усеченного Ньютона).
- Линейная алгебра алгоритма: внутренний цикл CG, двухцикловая рекурсия L-BFGS и т.д.
- Линейный поиск: вызовы функций внутри условий Вульфа.

2.5.3 Вопросы для анализа в отчете:

1. У какого метода линейный поиск «съедает» самую большую долю времени?

2. Почему на задачах с $n \gg m$ усеченный Ньютон может проигрывать L-BFGS по времени?

2.6 Эксперимент: Оптимизационная точность против качества предсказания

В реальных задачах целевая функция, которая оптимизируется методами оптимизации, как правило, не является конечным критерием качества решения задачи. Например в задачах машинного обучения нашей конечной целью является не минимизация функции потерь $f(x)$ на обучающей выборке до машинного нуля, а получение модели с высокой обобщающей способностью (минимальной ошибкой на тестовых данных). Существует порог оптимизационной точности ϵ, после которого дальнейшее уменьшение значения функции потерь при обучении уже не приводит к улучшению метрик на тестовом множестве.

2.6.1 Используемые функции:

Ваши индивидуальные ML-оракулы (задача классификации и задача регрессии) на соответствующих датасетах. Обязательно разделите данные на обучающую (train) и тестовую (test) выборки в пропорции 80/20.

2.6.2 Порядок проведения

Возьмите любой метод оптимизации (например, L-BFGS) и запустите его (на обучающей выборке) с высокой точностью $\epsilon = 10^{-8}$. [3].

[3] Напомним, что во всех наших методах оптимизации $\epsilon \in (0, 1)$ задает относительную точность по квадрату нормы градиента: $\|\nabla f(\hat{x})\|^2 \leq \epsilon \|\nabla f(x_0)\|^2$.

На каждой итерации алгоритма k фиксируйте три величины:

- Значение целевой функции на обучающем множестве $f_{train}(x_k)$
- Норму градиента $||\nabla f(x_k)||_2$
- Метрику качества на тестовой выборке (Accuracy (*доля правильно классифицированных объектов от общего количества объектов в тестовом наборе данных*) для классификации и MSE/MAE для регрессии)

Постройте график с двумя осями Y: - Левая ось Y (логарифмическая): Значение $f_{train}(x_k)$ и $||\nabla f(x_k)||$ от номера итерации (или от времени).

- Правая ось Y (линейная): Тестовая метрика качества от номера итерации (или от времени).

2.6.3 Вопросы для анализа в отчете:

1. На какой итерации (или при какой норме градиента) метрика качества на тесте перестает значимо улучшаться? Совпадает ли этот момент с «визуальной» сходимостью функции потерь?
2. Какую долю общего времени работы алгоритма занимает «бесполезная» оптимизация (когда f_{train} еще падает, а точность на тесте уже вышла на плато)? Какое ϵ стоит использовать для реальных задач?
3. Как меняется «точка насыщения» при изменении параметра λ? Становится ли разрыв между обучением и тестом больше при очень маленьких λ?

2.7 Углубленное исследование: Индивидуальный трек команды

В данном разделе команда должна представить результаты исследования по своему индивидуальному треку (см. Приложение А).

Отчет по треку должен содержать:

1. Постановку задачи: Что именно исследовалось.
2. Описание эксперимента: Какие параметры варьировались, на каких данных проводились тесты.
3. Результаты: Графики, таблицы, профили времени работы.
4. Выводы: Глубокая аналитика полученных результатов. Почему метод вел себя именно так? Подтверждают ли результаты теорию из лекций?

3 Оформление задания

Работа команды сдается в виде трех взаимосвязанных артефактов (см. Лабораторная работа 1):

- Репозиторий на Github/Gitlab
- Отчет в PDF
- Презентация

Каждый проведенный эксперимент следует оформить как отдельный раздел в PDF документе (название раздела - название соответствующего

эксперимента). Для каждого эксперимента необходимо сначала написать его описание:

а) постановка задачи

б) какие функции оптимизируются, каким образом генерируются данные, Описание датасетов (размерность $m \times n$, плотность/разреженность матриц), используемого "железа"(CPU/RAM), какие методы и с какими параметрами используются.

в) далее должны быть представлены результаты соответствующего эксперимента - графики, таблицы и т. д.

г) наконец, после результатов эксперимента должны быть написаны Ваши выводы - какая зависимость наблюдается и почему. Как это бьется с теорией оптимизации из лекций?

Важно: Отчет не должен содержать никакого кода. Каждый график должен быть прокомментирован - что на нем изображено, какие выводы можно сделать из этого эксперимента. Обязательно должны быть подписаны оси. Если на графике нарисовано несколько кривых, то должна быть легенда. Сами линии следует рисовать достаточно толстыми, чтобы они были хорошо видимыми.